

Righteous IT *Join the crusade!*

Linux Investigation (Part 2)

Posted on ~~May 5, 2026~~ April 29, 2026 by Hal Pomeranz

Please read the previous installment (<https://righteousit.com/2026/05/04/linux-investigation-part-1/>) of this investigation for additional background.

Having identified a potential LD_PRELOAD rootkit, I'm very curious to discover what the memory dump from the system can tell us. Note that before you can begin analyzing the memory for yourself you will need to (a) install Volatility (<https://github.com/volatilityfoundation/volatility3>), and (b) install a Linux profile (https://github.com/Abyss-W4tcher/volatility3-symbols/blob/master/Debian/amd64/6.12.74%2Bdeb13%2B1/Debian_6.12.74%2Bdeb13%2B1-amd64_6.12.74-2_amd64.json.xz) that will work for this memory dump in `.../volatility3/symbols/linux`.

Process Information

One approach for finding suspicious processes is to look at the process hierarchy with `linux.pstree`. The Linux process hierarchy is usually rather flat, so interactive sessions tend to stand out:

```
(venv) $ vol -q -f memory_dump/avml.lime linux.pstree
[...]
```

* 0x8c7cc67e1980	937	937	1	sshd
** 0x8c7cc8278000	939	939	937	sh
*** 0x8c7cc67e4c80	940	940	939	python3
**** 0x8c7cc6011980	941	941	940	bash
***** 0x8c7cc6014c80	975	975	941	ssh
** 0x8c7cc81b1980	1005	1005	937	sshd-session
*** 0x8c7cc08b8000	1047	1047	1005	sshd-session
**** 0x8c7cc8354c80	1064	1064	1047	bash
***** 0x8c7cc351b300	1076	1076	1064	sudo
***** 0x8c7cc3859980	1078	1078	1076	sudo
***** 0x8c7cc039cc80	1079	1079	1078	bash
***** 0x8c7da0131980	119319	119319	1079	avml

```
[...]
```

The second hierarchy starting with PID 1005 is what SSH sessions normally look like: two `sshd-session` processes (privilege separation), then the login shell. The double `sudo` is an interesting wrinkle, but apparently an artifact of this user doing “`sudo -s`” to start a root shell rather than just “`sudo bash`”. Then we have the `bash` shell itself, where the user is running `avml` to grab the memory dump we are currently reviewing. This is more or less as expected.

The process hierarchy starting with PID 939, however, is just plain weird. The shell process with PID 939 is spawned directly out of the master SSH daemon, PID 937. This suggests some sort of exploit against the SSH daemon itself. Next comes a Python process (PID 940). Could this be the Python command from the original SOC report? Certainly PID 940 spawned a `bash` shell (PID 941), which matches what the SOC told us. That `bash` shell ran “`ssh`”– the SSH client program. What is going on here?

We can get more detail on these processes from the `linux.psaux` plugin:

```
(venv) $ vol -q -f memory_dump/avml.lime linux.psaux
[...]
```

939	937	sh	/bin/sh
940	939	python3	python3 -c import pty; pty.spawn("/bin/bash")
941	940	bash	/bin/bash

```
[...]
```

975	941	ssh	ssh -F config ymv
-----	-----	-----	-------------------

```
[...]
```

That’s definitely the Python command line the SOC reported to us. Apparently whatever is happening in the PID 939 shell is not happening inside of an encrypted session. This also tends to point towards some sort of pre-encryption and therefore pre-authentication compromise of the master SSH daemon. Recall from our log analysis in the last installment that the `/etc/ld.so.preload` file containing the path of the suspected rootkit library was created immediately before the SSH daemon was restarted. Are we looking at rootkit functionality for the SSH daemon compromise?

It’s also worth considering that `ssh` command line (PID 975). “`-F config`” means read client options from a file called “`config`”, but where was that file dropped to disk? Also, the SOC and the system admins for the machine have no idea what the host “`ymv`” is. It does not resolve within this network. Best guess is that the hostname “`ymv`” refers to configuration in the “`config`” file, wherever that ended up.

Command History From Memory

UAC didn’t find command history on disk, but the attacker’s `bash` shell was still active at the time the memory image was taken. So that command history should be in the memory dump.

```
(venv) $ vol -q -f memory_dump/avml.lime linux.bash --pid 941
Volatility 3 Framework 2.27.1
```

PID	Process	CommandTime	Command
941	bash	2026-03-24 23:26:26.000000 UTC	rest
941	bash	2026-03-24 23:26:30.000000 UTC	reset
941	bash	2026-03-24 23:26:44.000000 UTC	echo \$SHELL
941	bash	2026-03-24 23:26:54.000000 UTC	id
941	bash	2026-03-24 23:27:25.000000 UTC	cd /dev/shm/kit
941	bash	2026-03-24 23:27:26.000000 UTC	ls
941	bash	2026-03-24 23:27:51.000000 UTC	mv xmrig top
941	bash	2026-03-24 23:27:59.000000 UTC	export PATH=.:\$PATH
941	bash	2026-03-24 23:28:17.000000 UTC	ssh -F config ymv
941	bash	2026-03-24 23:28:28.000000 UTC	bg
941	bash	2026-03-24 23:29:09.000000 UTC	top -o 127.0.0.1:3333 -B
941	bash	2026-03-24 23:29:15.000000 UTC	cd ..
941	bash	2026-03-24 23:29:19.000000 UTC	rm -rf kit
941	bash	2026-03-24 23:29:24.000000 UTC	ps -ef grep top
941	bash	2026-03-24 23:29:36.000000 UTC	ls

This shell history is extremely useful. After what is apparently an initial typo, the user executes a “reset” command, which is part of the [typical recipe \(https://blog.ropnop.com/upgrading-simple-shells-to-fully-interactive-ttys/\)](https://blog.ropnop.com/upgrading-simple-shells-to-fully-interactive-ttys/) for elevating a raw shell to fully interactive via the Python `pty.spawn()` method. Our user then checks which shell they are in (“echo \$SHELL”) and what user they are running as (“id”). This is classic post-exploit behavior: a typical user in a normal interactive session would not need to run these commands.

The user then changes to a directory named `/dev/shm/kit`. This is not a typical directory on this system, so where did it come from? The fact that it was staged in `/dev/shm`—a memory-based file system—is suspicious. This would be a good place for an attacker to stage files that they did not want to write to the system’s local disk.

Inside this directory was apparently a file named “xmr`ig`”. XMRig is a popular Monero cryptocurrency miner. The user renames this file to “top” and later executes this “top” program from the `/dev/shm/kit` directory. “export PATH=.:\$PATH” adds the current working directory to the front of the search path, followed later by “top -o 127.0.0.1:3333 -B” to execute the program. Note that the command line arguments to this “top” program match typical xmr`ig` arguments—“-o 127.0.0.1:3333” to specify the mining infrastructure to connect to, and “-B” to run in the background as a daemon.

After starting the renamed xmr`ig`, we see the user remove `/dev/shm/kit` (“cd ..”, “rm -rf kit”). This will not impact the running processes that were started here, but will make it more difficult for inexperienced investigators to find these files. We also see the user checking that their “top” program is still running or perhaps that it is invisible (“ps -ef | grep top”), and making sure that `/dev/shm/kit` is gone (“ls”).

But what is happening on localhost 3333/tcp? In between the “mv” command to rename the binary and executing the renamed xmr`ig` as “top” we see “ssh -F config ymv” (later set to run in the background with “bg”) which we also saw in the `linux.psaux` output. Note that this command history implies that the “config” file was in `/dev/shm/kit`, now deleted.

It seems reasonable to assume that 127.0.0.1:3333 is an SSH tunnel. But can we find artifacts in memory to prove that?

Network Details

The `linux.sockstat` plugin should give us some insight into the network behavior on the system:

```
(venv) $ vol -q -f memory_dump/avml.lime linux.sockstat | grep -F 3333 |
sort -u
4026531840      libuv-worker      977      979      17      0x8c7cc405c280
AF_INET STREAM TCP      127.0.0.1      59182    127.0.0.1      3333
ESTABLISHED    -
4026531840      libuv-worker      977      980      17      0x8c7cc405c280
AF_INET STREAM TCP      127.0.0.1      59182    127.0.0.1      3333
ESTABLISHED    -
4026531840      libuv-worker      977      981      17      0x8c7cc405c280
AF_INET STREAM TCP      127.0.0.1      59182    127.0.0.1      3333
ESTABLISHED    -
4026531840      libuv-worker      977      982      17      0x8c7cc405c280
AF_INET STREAM TCP      127.0.0.1      59182    127.0.0.1      3333
ESTABLISHED    -
4026531840      ssh      975      975      4      0x8c7cc404a900 AF_INET6
STREAM TCP    ::1      3333      ::      0      LISTEN -
4026531840      ssh      975      975      5      0x8c7cc4043900 AF_INET
STREAM TCP    127.0.0.1      3333    0.0.0.0 0      LISTEN -
4026531840      ssh      975      975      6      0x8c7cc405e880 AF_INET
STREAM TCP    127.0.0.1      3333    127.0.0.1      59182
ESTABLISHED    -
4026531840      top      977      977      17      0x8c7cc405c280 AF_INET
STREAM TCP    127.0.0.1      59182    127.0.0.1      3333
ESTABLISHED    -
4026531840      top      977      978      17      0x8c7cc405c280 AF_INET
STREAM TCP    127.0.0.1      59182    127.0.0.1      3333
ESTABLISHED    -
4026531840      top      977      987      17      0x8c7cc405c280 AF_INET
STREAM TCP    127.0.0.1      59182    127.0.0.1      3333
ESTABLISHED    -
4026531840      top      977      988      17      0x8c7cc405c280 AF_INET
STREAM TCP    127.0.0.1      59182    127.0.0.1      3333
ESTABLISHED    -
4026531840      top      977      989      17      0x8c7cc405c280 AF_INET
STREAM TCP    127.0.0.1      59182    127.0.0.1      3333
ESTABLISHED    -
4026531840      top      977      990      17      0x8c7cc405c280 AF_INET
STREAM TCP    127.0.0.1      59182    127.0.0.1      3333
ESTABLISHED    -
```

The suspicious SSH process that was started (PID 975) is definitely listening on 127.0.0.1:3333. There is no “-L” option showing on the command line, so we presume the tunnel configuration is in the “config” file mentioned on the command line (“-F config”).

We can also see the “top” process, which is the renamed `xmrig` binary, actively talking on this port. This is consistent with the “-o 127.0.0.1:3333” option we saw in the command history. XMRig uses `libuv` for thread management, which is where the `libuv-worker` processes come from. Note that the `top` and `libuv-worker` entries share the same PID (977 in the third column of output) but different thread IDs (fourth column of output above).

But where is the SSH process connecting to? Let’s check the `linux.sockstat` output again:

```
(venv) $ vol -q -f memory_dump/avml.lime linux.sockstat | awk '$3 == "975"'
```

4026531840	ssh	975	975	3	0x8c7cc405cc00	AF_INET
STREAM TCP	192.168.4.22	33440	192.168.5.95	22		
ESTABLISHED	-					
4026531840	ssh	975	975	4	0x8c7cc404a900	AF_INET6
STREAM TCP	::1	3333	::	0	LISTEN	-
4026531840	ssh	975	975	5	0x8c7cc4043900	AF_INET
STREAM TCP	127.0.0.1	3333	0.0.0.0	0	LISTEN	-
4026531840	ssh	975	975	6	0x8c7cc405e880	AF_INET
STREAM TCP	127.0.0.1	3333	127.0.0.1	59182		
ESTABLISHED	-					

The outbound connection is to 192.158.5.95. We understood from the initial SOC report that all access to the system we are investigating is supposed to go through a jump host at 192.168.4.35, so this is a new system we have not heard of. The IP address is internal, and we will need to investigate this system to find out whether it too has been compromised. The scope of our investigation is growing!

We can also use `linux.sockstat` to investigate where our mysterious unencrypted SSH traffic is originating from. Here I am focusing in on only the activity related to the initial “sh” process (PID 939) that was created:

```
(venv) $ vol -q -f memory_dump/avml.lime linux.sockstat | awk '$3 == "939"'
```

4026531840	sh	939	939	0	0x8c7cc4059300	AF_INET
STREAM TCP	192.168.4.22	22	192.168.4.35	48411		
ESTABLISHED	-					
4026531840	sh	939	939	1	0x8c7cc4059300	AF_INET
STREAM TCP	192.168.4.22	22	192.168.4.35	48411		
ESTABLISHED	-					
4026531840	sh	939	939	2	0x8c7cc4059300	AF_INET
STREAM TCP	192.168.4.22	22	192.168.4.35	48411		
ESTABLISHED	-					
4026531840	sh	939	939	8	0x8c7cc4059300	AF_INET
STREAM TCP	192.168.4.22	22	192.168.4.35	48411		
ESTABLISHED	-					

It appears that this connection originated from the jump host system, 192.168.4.35 (source port 48411).

Detailed Process Information

The `linux.pslist` plugin will give us process start times as well as UID and GID info for all of our suspicious processes. The `linux.pslist` output is quite busy, so allow me to summarize the important pieces of info:

Process	PID	PPID	UID	GID	Started (UTC)
sh	939	937	0	7823	2026-03-24 23:26:07
python3	940	939	0	7823	2026-03-24 23:26:22
bash	941	940	0	7823	2026-03-24 23:26:22
ssh	975	941	0	7823	2026-03-24 23:28:32
top	977	1	0	7823	2026-03-24 23:29:24

One item that jumps out is that all of the processes are running as `root` (UID zero), but with the GID 7823. No other processes in the output are using this GID and the GID does not appear in the `/etc/group` file captured by UAC (`[root]/etc/group`). Rootkits will often use a group ID value to mark processes, files, and directories that should be hidden by the rootkit. That may be what is happening here.

Also note that the PPID of the “`top`” process is 1, which is `systemd`. This is an artifact of the “`-B`” option that was used to invoke the program. This option tells the process to run in the background like any other daemon process. In doing so the “`top`” process disassociates itself from the parent shell that spawned it, inheriting PPID 1. This is also why the “`top`” process did not appear in `linux.pstree` process hierarchy while the “`ssh`” command did.

Note that the start times for the “`ssh`” and “`top`” processes reported by `linux.pslist` differ from the timestamps from the command history. The `linux.pslist` times are approximately 15 seconds later than the corresponding command history timestamps. I have no explanation for this discrepancy, but will stick with the timestamps from the command history for consistency.

Status Check And Next Steps

Our timeline is filling in nicely. I’ve added a note at the end of each item to remind us where the information comes from:

```
23:22:19    User "worker" logs in with password from jump host
(192.168.4.35) port 48364 [logs]
23:23:34    User "worker" uses sudo to execute root shell [logs]
23:23:48    Command-only/scp as user "worker" from jump host port 55504
[logs]
23:24:51    /usr/lib/x86_64-linux-gnu/libymv.so.3 created [bodyfile]
23:25:09    /etc/ld.so.preload created, points to .../libymv.so.3
[chkrootkit]
23:25:19    SSH daemon restarted [logs]
23:26:07    Unencrypted connect from 192.168.4.35 port 48411 spawns hidden
sh (PID 939) [memory]
23:26:22    python3 execution to promote raw shell [memory]
23:26:22    Hidden bash process started from python pty.spawn() [memory]
23:27:16    User "worker" SSH session from jump host port 48364 ends
[logs]
23:27:51    /dev/shm/kit/xmrig renamed to "top" [memory]
23:28:17    ssh to 192.168.5.95 with tunnel on 3333/tcp [memory]
23:29:09    "top" process (renamed xmrig) started, comms via SSH tunnel
[memory]
23:29:19    /dev/shm/kit removed [memory]
```

From the timeline it appears that the suspicious library was planted during the original “worker” login session that started at 23:22:19. But once the hidden session was established and promoted via `pty.spawn()`, the legitimate login closed down. The hidden session was responsible for starting the SSH tunnel to 192.168.5.95 and running the renamed XMRig.

But there are still so many questions. How did the `/dev/shm/kit` directory and the suspicious library get onto the system in the first place and when? Can we recover any of the suspect files—“`libymv.so.3`”, “`top`”, and the “`config`” file used for the SSH connection? Is “`libymv.so.3`” really an LD_PRELOAD rootkit as suspected? What else did the attacker accomplish on the system? More investigation in the next installment!

Posted in [DFIR](#), [Forensics](#), [Linux](#) Tagged [Contest](#), [DFIR](#), [Forensics](#), [Linux](#), [Scenario](#)



Published by Hal Pomeranz

Independent Computer Forensics and Information Security consultant. Expert Witness. Trainer. [View all posts by Hal Pomeranz](#)

[Blog at WordPress.com.](#)